

CAMPUS ISSUE RAISER - TECHNICAL DEEP DIVE REPORT

0. Document Purpose

This document provides a code-backed technical report of the CAMPUS ISSUE RAISER project. It is written for academic review, engineering documentation, and viva/interview defense where implementation detail is expected, not just feature-level explanation.

The repository contains two backend implementations:

1. Active runtime system: microservices in `services/` + API gateway + React frontend.
2. Legacy implementation: monolithic Express + Mongoose code in `backend/`.

All runtime commands (`npm run dev`, `docker compose up`) target the microservice stack.

1. System Topology and Runtime Architecture

1.1 Active services and network layout

The active architecture is a gateway-mediated multi-service backend:

1. API Gateway (`services/gateway/server.js`) - port `5000`
2. User Service (`services/user-service/server.js`) - port `5001`
3. Section Service (`services/section-service/server.js`) - port `5002`
4. Issue Service (`services/issue-service/server.js`) - port `5003`
5. Interaction Service (`services/interaction-service/server.js`) - port `5004`
6. Evidence Service (`services/evidence-service/server.js`) - port `5005`
7. Escalation Service (`services/escalation-service/server.js`) - port `5006`
8. Status Service (`services/status-service/server.js`) - port `5007`
9. Analytics Service (`services/analytics-service/server.js`) - port `5008`

Frontend (`frontend/`) runs on Vite and talks only to gateway `/api/\*`.

1.2 Why gateway-centric routing

The gateway centralizes:

1. Client-facing API base URL
2. Route fan-out to internal services
3. Header forwarding (including bearer token)
4. Failure normalization (`502` when service unreachable)

This keeps frontend integration stable even if service internals evolve.

1.3 Practical request flow

Example: create issue

1. Browser -> `POST /api/issues` (gateway)
 2. Gateway proxies -> issue-service `/issues`
 3. Issue-service may call user-service `/users/authorities`
 4. Issue persisted in shared store
 5. Optional notification pushed through user-service internal endpoint
 6. Response propagates back through gateway to frontend
-

2. API Gateway Deep Dive

Source: ``services/gateway/server.js``

2.1 Route dispatch matrix

Gateway route behavior is explicit and ordered:

1. ``/api/auth/*`` -> user-service
2. ``/api/users/*`` -> user-service
3. ``/api/sections/*`` -> section-service
4. ``/api/interactions/*`` -> interaction-service
5. ``/api/evidence/*`` -> evidence-service
6. ``/api/analytics/*`` -> analytics-service
7. ``/api/issues/:id/status`` -> status-service
8. ``/api/issues/:id/escalate`` -> escalation-service
9. ``/api/issues/*`` -> issue-service catch-all

Route ordering matters because specific issue routes must be matched before the issue catch-all route.

2.2 Proxy semantics

Gateway forwards:

1. HTTP method
2. Query string
3. JSON body (if present)
4. Authorization header unchanged

Token verification is not done at gateway layer; each service independently validates JWT via shared middleware.

2.3 Health endpoint

``GET /health`` is exposed by gateway and used as a lightweight liveness check.

3. Shared Infrastructure Modules (``services/common``)

3.1 ``auth.js`` - JWT and role authorization internals

Token issuance

``signToken(user)`` signs payload:

1. ``_id``
2. ``name``
3. ``email``
4. ``role``
5. ``department``

with:

1. `secret`process.env.JWT_SECRET || 'dev-secret``
2. `expiry`7d``

Authentication middleware

``auth`` middleware:

1. Reads ``Authorization`` header
2. Expects ``Bearer <token>`` format
3. Verifies token with ``jsonwebtoken.verify``

4. Attaches decoded payload to `req.user`
5. Returns `401` for missing/invalid token

Authorization middleware

`authorize(roles)` middleware:

1. Checks `req.user.role`
2. Returns `403` if role not allowed

This creates a clear split between identity verification (`auth`) and privilege control (`authorize`).

3.2 `store.js` - dual persistence abstraction

The project uses one storage interface supporting two modes.

File mode (`STORE_MODE=file`, default)

1. JSON files under `services/data`
2. `readJson` and `writeJson` use filesystem operations
3. In-memory cache (`Map`) avoids repeated disk parsing

Mongo mode (`STORE_MODE=mongo` or `MONGO_URI` set)

1. Single collection model storing `key -> data`
2. Keys map to logical files (for example `issues.json`)
3. Cache remains primary read source
4. Updates upsert into Mongo

Core behavior

1. `updateJson(fileName, defaultData, updaterFn)` provides atomic-style read-modify-write at service level
2. Writes update cache first, then persistence backend
3. Returned objects are cloned to avoid accidental mutation leaks

This allows the same service logic to run in local demo mode and DB-backed mode.

3.3 `http.js` - inter-service communication helper

Services use shared HTTP helper wrappers for internal REST calls. This standardizes JSON request handling and reduces repetitive fetch boilerplate.

3.4 `logger.js` - operational audit log

Services append structured entries to `logs.json` via common logger utility. Log consumption is exposed to admin through user-service endpoint.

4. Service Contracts and Business Logic

4.1 User Service (`services/user-service/server.js`)

Responsibilities

1. Registration and login
2. Current user lookup (`/auth/me`)
3. Authority discovery for assignment/escalation
4. Tabable authority list for tagging
5. User directory for UI name resolution
6. Notification inbox operations
7. Admin access to audit logs

Endpoint specification

1. `POST /auth/register``
 - Required: ``name``, ``email``, ``password``
 - Email normalized to lowercase
 - Password hashed with bcrypt
 - Role defaults to ``student``
2. `POST /auth/login``
 - Required: ``email``, ``password``
 - Password verified with bcrypt compare
 - Returns token + user projection
3. `GET /auth/me`` (auth required)
 - Returns current user from token identity
4. `GET /users/authorities?level=&department=``
 - Maps escalation level to authority role
 - Supports department filtering
5. `GET /users/taggable`` (auth required)
 - Returns non-student authorities usable in issue tagging
6. `GET /users/directory`` (auth required)
 - Returns lightweight user list for UI lookups
7. `GET /users/notifications`` (auth required)
 - Returns recipient-specific notifications sorted newest first
8. `PATCH /users/notifications/:id/read`` (auth required)
 - Marks a notification as read if owned by caller
9. `POST /users/notifications/read-all`` (auth required)
 - Bulk mark-read for current user
10. `POST /notifications/internal``
 - Internal creation endpoint used by services
11. `GET /users/logs`` (auth required, admin only)
 - Returns centralized logs

Technical notes

1. Role mapping is core to escalation chain (``teacher` -> `hod` -> `dean` -> `management`)`
 2. Notification model stores read-state and recipientId
 3. User directory endpoint avoids expensive joins by delegating display-name mapping to frontend
-

4.2 Issue Service (`services/issue-service/server.js``)

Responsibilities

1. Issue creation
2. Issue retrieval with filters
3. Single issue retrieval
4. Status mutation endpoint for status-service
5. Support-count patch endpoint for interaction-service
6. Escalation patch endpoint for escalation-service

Data model fields

Typical issue object fields:

1. ``_id``

2. ``title``
3. ``description``
4. ``sectionId``
5. ``tags[]``
6. ``taggedAuthorityIds[]``
7. ``imageUrl``
8. ``isAnonymous``
9. ``createdBy``
10. ``createdByDepartment``
11. ``assignedTo``
12. ``escalationLevel``
13. ``status``
14. ``supportCount``
15. ``createdAt``

Endpoint specification

1. ``POST /issues`` (auth required)
 - Required: ``title``, ``description``, ``sectionId``
 - Creates default state: status ``Open``, escalation level ``1``
 - Auto-assigns authority by calling user-service authorities endpoint
 - Optionally creates internal notifications for tagged authorities
2. ``GET /issues`` (auth required)
 - Filter support: section/status/tag
 - Sort: reverse chronological
 - Applies anonymity masking policy for unauthorized viewers
3. ``GET /issues/:id`` (auth required)
 - Returns single issue with same anonymity policy
4. ``PATCH /issues/:id/status``
 - Internal endpoint for status-service
 - Valid status set enforced
5. ``PATCH /issues/:id/support``
 - Internal endpoint for interaction-service
 - Applies delta update to supportCount (minimum zero)
6. ``PATCH /issues/:id/escalate``
 - Internal endpoint for escalation-service
 - Mutates escalation level, assignee, and status

Technical notes

1. Service owns canonical issue state
2. Other services mutate only via controlled internal patch endpoints
3. Department-aware authority assignment occurs at create time

4.3 Interaction Service (`services/interaction-service/server.js``)

Responsibilities

1. Support reaction handling
2. Comment creation
3. Interaction listing per issue

Endpoint specification

1. `POST /interactions/support`` (auth required)
 - Required: ``issueld``
 - Duplicate supports blocked per user/issue pair
 - On success, calls issue-service support patch endpoint
2. `POST /interactions/comment`` (auth required)
 - Required: ``issueld``, ``content``
 - Content normalized by trimming
3. `GET /interactions/:issueld`` (auth required)
 - Returns interactions linked to issue

Technical notes

1. Support operation is idempotent from user perspective (cannot support twice)
 2. Interaction record and issue aggregate count are updated through coordinated service calls
-

4.4 Evidence Service (`services/evidence-service/server.js``)

Responsibilities

1. Accept issue evidence
2. List evidence per issue

Endpoint specification

1. `POST /evidence`` (auth required)
 - Required: ``issueld``
 - At least one of ``text`` or ``fileUrl`` must be present
2. `GET /evidence/:issueld`` (auth required)
 - Returns issue evidence collection

Technical notes

Evidence is modeled independently from comments to keep moderation and retrieval concerns separate.

4.5 Escalation Service (`services/escalation-service/server.js``)

Responsibilities

1. Controlled escalation decisions
2. Assignee rotation to next authority tier

Endpoint specification

1. `PUT /issues/:id/escalate`` (auth required)
 - Reads issue from issue-service
 - Rejects escalation for resolved issues
 - Prevents no-op reassignment to same user
 - Computes ``nextLevel = min(4, current + 1)``
 - Fetches new authority from user-service
 - Persists via issue-service escalate patch endpoint

Technical notes

Escalation policy is encoded in dedicated service rather than issue-service to isolate progression logic and allow future SLA policy expansion.

4.6 Status Service (`services/status-service/server.js`)

Responsibilities

1. Admin-only issue status transitions

Endpoint specification

1. `PUT /issues/:id/status` (auth required)
 - Enforces role `admin`
 - Validates allowed status enum
 - Persists through issue-service status patch endpoint

Technical notes

Keeping status writes in a separate service makes privilege boundaries explicit and auditable.

4.7 Section Service (`services/section-service/server.js`)

Responsibilities

1. Expose section taxonomy used by issue creation/filtering

Endpoint specification

1. `GET /sections`

Technical notes

Static seeded sections are sufficient for prototype scope; service boundary allows future CRUD and role-restricted taxonomy management.

4.8 Analytics Service (`services/analytics-service/server.js`)

Responsibilities

1. Aggregate weekly issue trends
2. Compute section and department performance

Endpoint specification

1. `GET /analytics/weekly` (auth required)

Response includes:

1. Totals (`totalIssues`, `resolved`, `pending`)
2. `issuesPerSection` map
3. `departmentPerformance` rows
4. Top and lowest department indicators
5. Generation timestamp

Scoring model

Example formula implemented in service:

1. `resolutionRate = resolved / complaints \* 100`
2. `score = resolved \* 3 - pending \* 2`
3. rating thresholds produce labels such as `Good`, `Average`, `Needs Attention`

Technical notes

Analytics are computed at request time from source datasets, suitable for current scale and demo usage.

5. Frontend Technical Architecture

Primary frontend files are under `frontend/src`.

5.1 Routing (`App.jsx` + route guards)

Route control is split into reusable wrappers:

1. `ProtectedRoute` - requires authenticated session
2. `AdminRoute` - requires authenticated admin role
3. `PublicOnlyRoute` - blocks login/register when already authenticated
4. `RouteLoader` - shown while auth bootstrap is unresolved

5.2 Session lifecycle (`context/AuthContext.jsx`)

State tracks:

1. token
2. user
3. authReady
4. isAuthenticated

Initialization sequence:

1. read token from localStorage
2. call `/auth/me` to rehydrate user
3. set `authReady` and route accordingly

Exposed actions:

1. `setSession(token)`
2. `clearSession()`
3. `refreshUser()`

5.3 API client (`services/api.js`)

Axios instance uses gateway base URL and request interceptor to attach bearer token automatically from localStorage.

This avoids repeating auth header logic in page components.

5.4 Page-level behavior details

1. `LoginPage.jsx`
 - Submits credentials to `/auth/login`
 - On success stores token and redirects to home
2. `RegisterPage.jsx`
 - Submits registration to `/auth/register`
 - Session bootstrapped immediately after signup
3. `CreateIssuePage.jsx`
 - Fetches sections and taggable authorities
 - Supports tags, authority tagging, anonymity flag
 - Handles optional image conversion to base64 with size checks
4. `IssuesPage.jsx`
 - Loads issue list and applies filters
 - Navigates to issue detail
5. `IssueDetailPage.jsx`
 - Loads issue + interactions + evidence + user directory
 - Supports comment submission, evidence submission, support action, escalate action
 - Admin user can trigger status update

6. `DashboardPage.jsx`
 - Pulls weekly analytics payload
 - Renders performance summaries and aggregates
 7. `AdminPage.jsx`
 - Command center for admin actions (status workflow and management screens)
 8. `ProfilePage.jsx`
 - Displays current user context and personal summary data
-

6. Data, Storage, and Consistency Model

6.1 Logical datasets

Shared datasets include:

1. `users.json`
2. `sections.json`
3. `issues.json`
4. `interactions.json`
5. `evidence.json`
6. `notifications.json`
7. `logs.json`

6.2 Single-writer principle per domain

1. User-service owns users/notifications semantics
2. Issue-service owns issue canonical state
3. Interaction-service owns reaction/comment records
4. Evidence-service owns evidence records

Cross-domain writes happen through explicit internal endpoints, not direct file access across services.

6.3 Consistency characteristics

1. Immediate consistency within single service write path
2. Eventual consistency across services for chained operations
3. In-memory caching improves read speed but means runtime process memory participates in state propagation

6.4 File mode vs Mongo mode behavior

1. File mode prioritizes demo simplicity
 2. Mongo mode improves persistence durability and multi-instance suitability
 3. Same service APIs and business logic are reused in both modes
-

7. Authentication, Authorization, and Security Posture

7.1 Implemented controls

1. Password hashing with bcrypt
2. JWT signed session tokens
3. Route-level auth middleware on protected endpoints
4. Role enforcement for admin-only operations
5. Input presence validation on core write endpoints

7.2 Notable enforcement points

1. Status update endpoint is admin-restricted

2. Support endpoint blocks duplicate support per user
3. Escalation flow blocks escalation on resolved issues
4. Evidence creation requires meaningful payload (`text` or `fileUrl`)

7.3 Security limitations (technical)

Current codebase remains prototype-grade in areas such as:

1. Missing request schema validation library integration
 2. No centralized rate-limiting middleware at gateway
 3. No refresh-token rotation strategy
 4. Limited hardening around internal service authentication
-

8. Operational Tooling and Environment

8.1 Root scripts (`package.json`)

1. `dev:full` - starts all services + gateway + frontend with concurrently
2. `dev:services` - backend stack without frontend
3. `dev:<service>` - single service execution for focused debugging
4. `seed:demo` - regenerate demo dataset
5. `seed:mongo` - import JSON seed data into Mongo mode
6. `export:deepdive:pdf` - render report markdown to PDF

8.2 Docker setup

From `Dockerfile` and `docker-compose.yml`:

1. Node runtime containerized
2. Mongo container included
3. Environment configured for `STORE\_MODE=mongo`
4. Ports mapped for gateway, all services, frontend, and Mongo

8.3 Local development workflow

Typical sequence:

1. `npm install`
 2. `npm --prefix frontend install`
 3. `npm run dev` (or split service/frontend terminals)
 4. Optional `npm run seed:demo`
-

9. Legacy Monolith (`backend/`) and Architectural Evolution

The `backend/` directory contains a monolithic Express + Mongoose design:

1. Controller-route-model layering
2. Legacy entities (college/group models)
3. Category and authority handling from earlier architecture phase

It is not part of current startup scripts or compose topology. Its presence is useful to explain engineering evolution from monolith to service decomposition.

Suggested viva framing:

1. Phase 1: monolith for fast feature iteration
 2. Phase 2: split into bounded services with gateway and shared infrastructure
 3. Outcome: clearer ownership boundaries and independently scalable components
-

10. End-to-End Technical Workflows

10.1 Issue creation and assignment

1. Authenticated user posts issue payload
2. Issue-service validates required fields
3. Issue-service requests authority candidates from user-service
4. Service assigns first matching level-1 authority
5. Issue persisted with default status and escalation level
6. Tagged authorities optionally notified

10.2 Support action workflow

1. User submits support for issue
2. Interaction-service checks duplicate support
3. Support interaction saved
4. Issue-service support count incremented via internal patch

10.3 Escalation workflow

1. User/admin triggers escalation endpoint
2. Escalation-service loads current issue state
3. Business rules validate escalation eligibility
4. Next-level authority resolved from user-service
5. Issue-service updates level, assignee, and escalated status

10.4 Status update workflow

1. Admin submits new status
2. Status-service validates role and status enum
3. Issue-service patch endpoint persists canonical status
4. Updated issue returned to client

11. Advanced Improvement Roadmap

Technical roadmap for production hardening:

1. Add API schema validation (for example Zod/Joi) per endpoint
2. Add centralized rate limit and abuse protection at gateway
3. Implement service-to-service trust model (internal JWT/mTLS)
4. Replace synchronous file writes with async transactional persistence
5. Add distributed tracing and correlation IDs across gateway/services
6. Add message queue for notification and analytics fan-out
7. Build stricter finite-state-machine transitions for issue status
8. Add test pyramid: unit tests for service logic, integration tests for gateway contracts, E2E tests for critical user journeys

12. Technical Conclusion

CAMPUS ISSUE RAISER is a working microservice-based complaint-resolution platform that demonstrates:

1. Bounded-domain service decomposition
2. Gateway-mediated API exposure
3. Shared authentication and storage abstractions
4. Role-aware workflows for escalation and status control

5. A functional React client integrated with secure token-based APIs

The implementation is suitable as a strong academic engineering project because it covers architecture, security, data modeling, service communication, frontend integration, and deployment tooling in one coherent system.